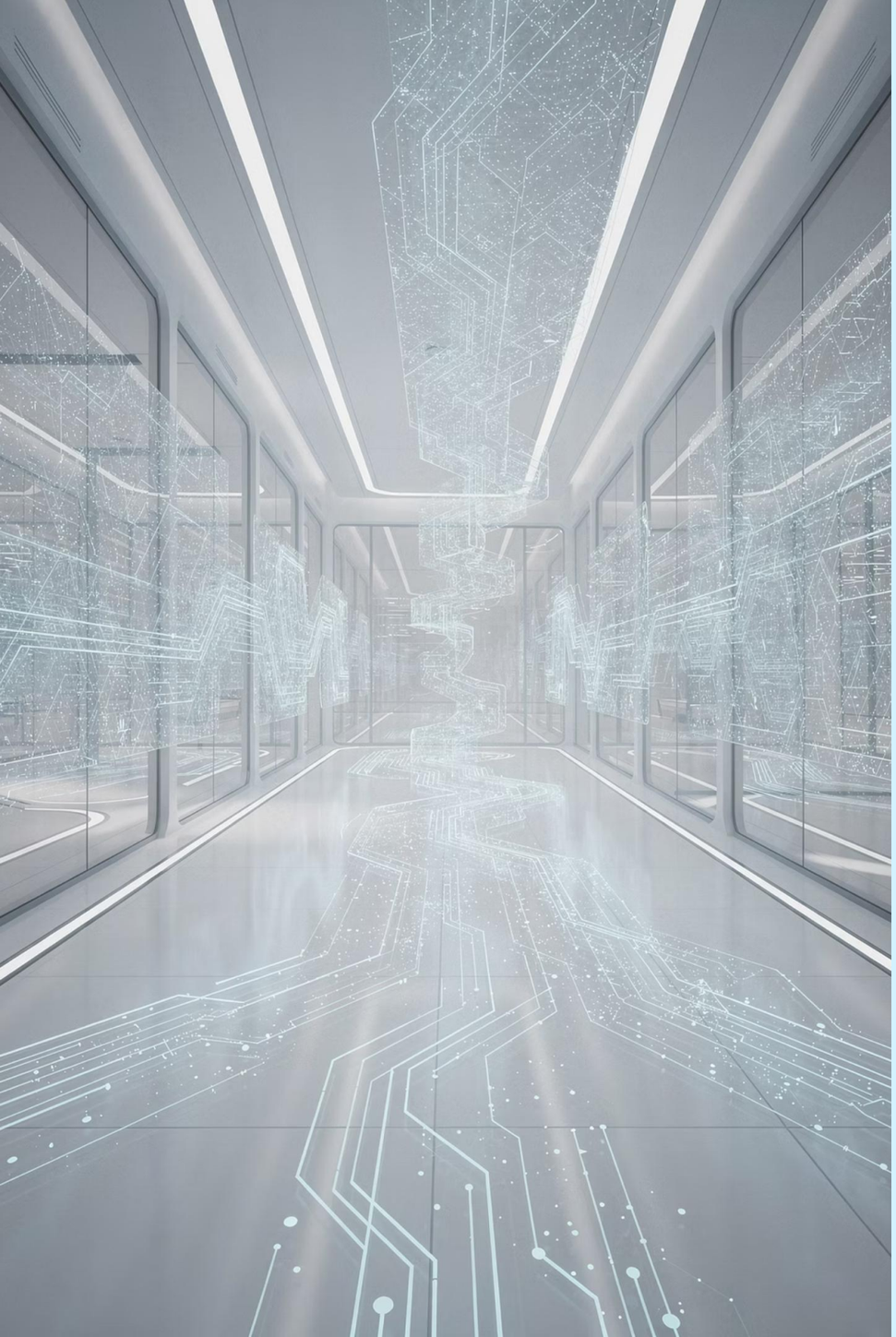
UNIWERSYTET
WSB MERITO
WROCŁAW

Python w analizie danych

- Kamil Musiał

- Wrocław, 2026



Dlaczego Dane Są Generowane Syntetycznie?

W tym projekcie dane są **sztucznie generowane**, a nie pobierane z prawdziwej firmy. To podejście ma wyraźne zalety dydaktyczne:

Dostępność

Każdy student uruchamia projekt bez szukania pliku — dane są od razu dostępne.

Elastyczność

Można łatwo modyfikować dane, wprowadzać błędy, braki i obserwacje odstające.

Realizm

Można symulować realne zależności biznesowe i skupić się na metodach analizy.

Import Bibliotek i Powtarzalność

Kluczowe biblioteki

- **os, math, random, warnings** — narzędzia systemowe
- **numpy** — obliczenia numeryczne
- **pandas** — dane tabelaryczne
- **matplotlib.pyplot** — wizualizacja
- **scikit-learn** — preprocessing, modelowanie, klasteryzacja

Dlaczego `random.seed(42)`?

Ustawienie ziarna losowości sprawia, że wyniki generowania danych są **powtarzalne**. Bez tego dwa uruchomienia programu mogłyby dawać zupełnie inne dane, wykresy i wyniki modeli.

W analizie danych powtarzalność eksperymentu jest niezwykle ważna.

Organizacja Projektu i Struktura Katalogów

Od początku program dba o **strukturę projektu**, definiując dedykowane ścieżki:

`DATA_DIR = "data"`

Dane wejściowe przechowywane osobno od wyników.

`OUTPUT_DIR = "output"`

Wyniki końcowe zapisywane w wydzielonym miejscu.

`PLOTS_DIR = output/plots`

Wykresy trafiają do osobnego folderu, gotowe do raportu.

W małych przykładach często zapisuje się wszystko „gdziekolwiek", ale w większej analizie szybko prowadzi to do bałaganu. To jest **bardzo dobra praktyka projektowa**.

Funkcje Pomocnicze

1

save_plot(filename)

Zapisuje wykres do PNG — wykresy nie giną po zamknięciu programu i trafiają do raportu.

2

print_section(title)

Czytelne oddzielenie sekcji w konsoli — poprawia czytelność pracy z programem.

3

generate_customer_id(index)

Tworzy identyfikatory w formacie `CUST_000123` — typowe w systemach biznesowych.

4

introduce_missing_values / introduce_outliers

Celowo wprowadzają **braki** i **obserwacje odstające** — bo prawdziwe dane są niekompletne i zawierają anomalie.

5

winsorize_series / calculate_rmse

Winsoryzacja „przycina" wartości do percentyli, zachowując rekordy. RMSE mierzy jakość modeli regresyjnych.

Generowanie Syntetycznego Zbioru Danych E-Commerce

Funkcja `generate_synthetic_ecommerce_data(n_rows=5000)` tworzy zbiór imitujący dane sklepu internetowego. Generowane cechy obejmują m.in.:

- identyfikator zamówienia i klienta
- datę zamówienia
- kategorię produktu i miasto
- wiek, płeć, dochód
- staż lojalnościowy
- liczbę wizyt na stronie
- rabat i liczbę sztuk
- cenę jednostkową i koszt dostawy
- metodę płatności
- informację o zwrocie i wartość zamówienia

Zależności Biznesowe w Danych

Kod nie losuje wszystkiego przypadkowo — wprowadza **logikę domenową**, która odzwierciedla realne zachowania rynkowe:

Elektronika vs. Książki

Elektronika ma wyższe ceny, książki są tańsze — zgodnie z realiami rynku.

Lojalność i Dochód

Wyższy dochód i większa lojalność klienta zwiększają wartość zakupów.

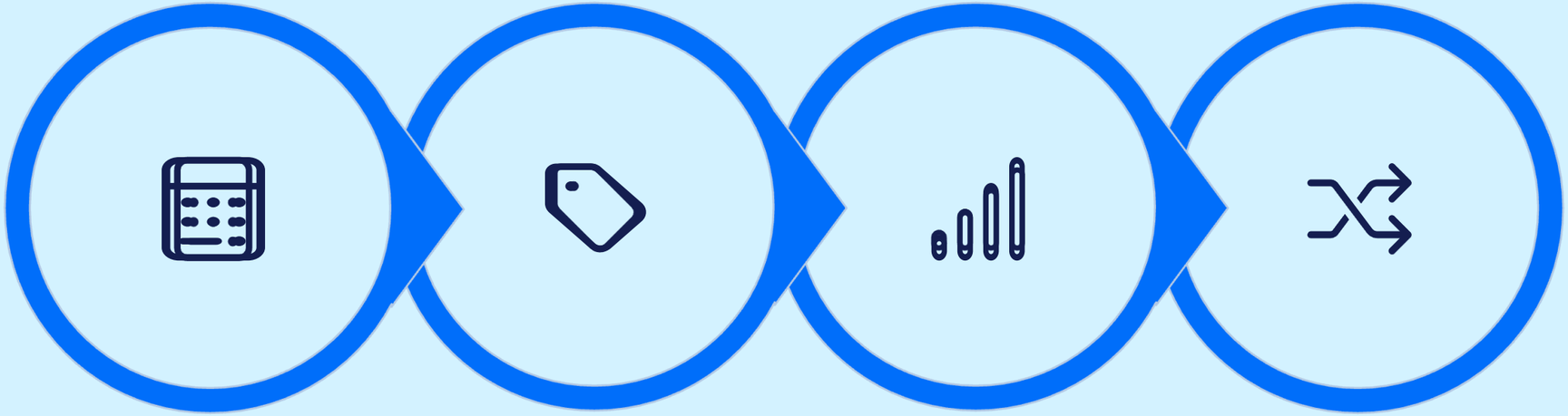
Ryzyko Zwrotów

Kategoria Fashion ma większe ryzyko zwrotu — realistyczna zależność branżowa.

Gdyby wszystko było losowe, modelowanie nie miałoby sensu. Dane analityczne powinny mieć logikę domenową.

Tworzenie Zmiennej Docelowej `order_value`

Wartość zamówienia jest obliczana krok po kroku, a nie wpisywana arbitralnie:



Wartość brutto → **Po rabacie** → **Efekt biznesowy** → **Wartość końcowa**

Dzięki temu dane są bardziej realistyczne, modele uczą się sensownych relacji, a analiza korelacji ma rzeczywiste znaczenie.

Wprowadzanie Braków i Outlierów

Po wygenerowaniu danych program **celowo** dodaje braki w wybranych kolumnach oraz obserwacje odstające. Dane od początku nie są idealne — i bardzo dobrze.

Brakujące wartości

Realne dane są niekompletne. Student uczy się nie tylko analizować „idealne" dane, ale też radzić sobie z brakami.

Obserwacje odstające

Outliery są częste w praktyce: błędnie wpisany dochód, anomalne zamówienie, rekord z nieprawidłową wartością.

Zapis i Odczyt Danych

create_and_save_dataset()

Generuje dane, zapisuje je do pliku CSV i zwraca DataFrame — jeden krok, kompletny wynik.

load_dataset(path)

Wczytuje dane z CSV z parametrem `parse_dates=["order_date"]` — kolumna daty jest od razu rozpoznawana jako typ daty, nie tekst.

Dzięki prawidłowemu parsowaniu dat można wygodnie wyciągać: **miesiąc, kwartał, dzień tygodnia** oraz informację, czy zamówienie było w weekend — co jest kluczowe dla analizy sezonowości.

Wstępna Inspekcja Danych

Funkcja `inspect_data(df)` to etap, którego początkujący często nie doceniają, a który jest **absolutnie kluczowy**.

→ **`head()` — pierwsze 5 rekordów**

Szybki podgląd struktury danych.

→ **`isna().sum()` — liczba braków**

Identyfikacja kolumn wymagających uzupełnienia.

→ **`info()` — typy i niepuste wartości**

Sprawdzenie typów kolumn i liczby braków.

→ **`describe()` — statystyki opisowe**

Zakres danych, średnie, odchylenia — pierwszy sygnał o podejrzanych wartościach.

Czyszczenie Danych

Funkcja `clean_data(df)` przygotowuje dane do dalszej pracy w sześciu etapach:

01

Usuwanie duplikatów

Powtórzone rekordy zaburzają wyniki — ta sama transakcja liczona dwa razy zawyża sprzedaż.

02

Uzupełnianie braków liczbowych

Dla `income`, `discount_percent`, `shipping_cost` stosowana jest **mediana** — odporna na skrajne wartości.

03

Uzupełnianie braków kategorycznych

Kolumna `gender` uzupełniana **modą** — najczęstszą wartością.

04

Korekta typów danych

Kolumny `age`, `quantity`, `returned` konwertowane do typów całkowitych.

05

Winsoryzacja i flagi binarne

Przycięcie ekstremów w `income`, `order_value`, `unit_price`. Dodanie flag `is_high_discount` i `is_returned`.

Inżynieria Cech

Funkcja `feature_engineering(df)` tworzy nowe zmienne — jeden z **najważniejszych etapów** analizy danych.

Cechy czasowe

Miesiąc, kwartał, dzień tygodnia, flaga weekendu — zachowania klientów zależą od czasu i sezonowości.

gross_value & discount_value

Wartość brutto (`cena × sztuki`) i rabat w złotówkach — 20% na laptopie to inna kwota niż na książce.

value_per_item

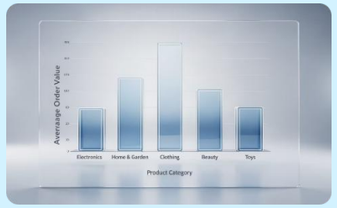
Średnia wartość na sztukę — pokazuje, czy klient kupuje dużo tanich czy mało drogich produktów.

Segmenty klientów

Wiek w grupach (18–25, 26–35, 36–50, 51–75) i dochód w kwartylach (low / medium / high / very_high).

Analiza Eksploracyjna Danych (EDA)

EDA to etap, w którym staramy się **zrozumieć dane** zanim przejdziemy do formalnych modeli. Funkcja `exploratory_data_analysis(df)` realizuje szereg analiz:



Wartość wg kategorii i miasta

Które grupy produktów i regiony generują największe zamówienia?

Sprzedaż miesięczna

Trendy, sezonowość i zmiany w czasie widoczne na wykresie liniowym.

Macierz korelacji

Zależności między zmiennymi — które cechy są silnie związane z `order_value`?

Analiza Statystyczna

Statystyki opisowe `order_value`

Położenie

Średnia i mediana opisują centrum rozkładu.

Rozproszenie

Odchylenie standardowe, Q1 i Q3 opisują zmienność.

Kluczowe analizy porównawcze

- **Grupy rabatowe:** porównanie średniej wartości zamówień dla rabatu ≥20% vs. <20%
- **Korelacje z order_value:** które cechy mają najsilniejszy związek z wartością zamówienia?

Korelacja nie oznacza przyczynowości — ale daje bardzo przydatny sygnał analityczny.

Modelowanie Regresyjne

Funkcja `build_regression_models(df)` buduje modele przewidujące wartość zamówienia. To klasyczny problem regresji:

Wejście

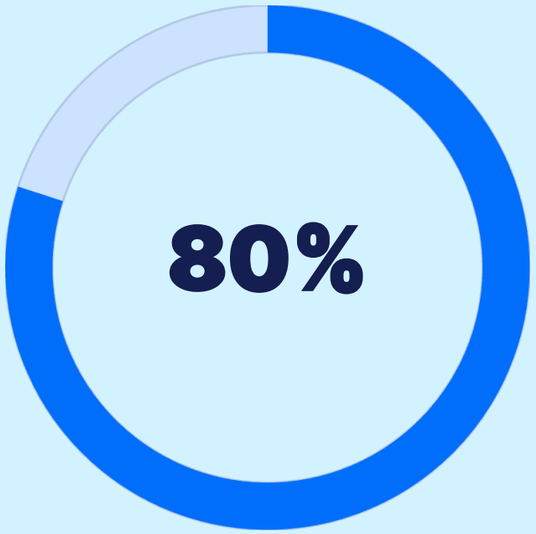
Cechy klienta i zamówienia — sensowne predyktory opisujące zjawisko.

Wyjście

Przewidywana wartość `order_value`.

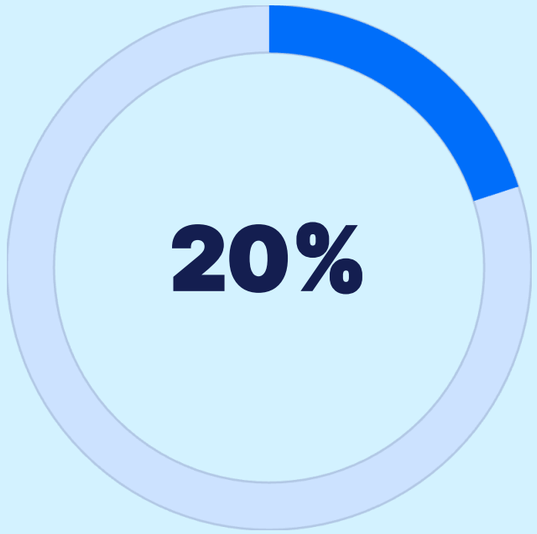
Program pomija `order_id` i `customer_id` — są to identyfikatory, a nie cechy opisujące zjawisko. **Model nie powinien uczyć się numerów rekordów.**

Podział na Zbiór Treningowy i Testowy



Dane treningowe

Na tych danych model się uczy zależności.



Dane testowe

Na tych danych uczciwie oceniamy jakość modelu.

Jeśli ocenialibyśmy model na tych samych danych, na których był uczony, wynik byłby zbyt optymistyczny. Podział jest konieczny dla rzetelnej ewaluacji.

Preprocessing Przed Modelem

To jedna z **najważniejszych części** całego programu — dane muszą być odpowiednio przygotowane przed podaniem do modelu.

Dane liczbowe

1. Imputacja medianą
2. Skalowanie cech

Skalowanie jest konieczne, bo dochód może mieć tysiące, a rabat tylko kilkanaście — modele są wrażliwe na skalę.

Dane kategoryczne

1. Imputacja najczęstszą wartością
2. One-Hot Encoding

OHE zamienia kategorie tekstowe na kolumny binarne. Np. `payment_method` daje kolumny: `_Card`, `_BLIK`, `_Transfer`. Model numeryczny nie pracuje bezpośrednio na tekście.

Pipeline i ColumnTransformer

To **nowoczesne i profesjonalne** podejście do organizacji preprocessingu i modelowania.

ColumnTransformer

Pozwala osobno przetwarzać kolumny liczbowe i kategoryczne w jednym kroku.

Pipeline

Łączy preprocessing z modelem w jeden spójny obiekt — mniej błędów, większa czytelność.

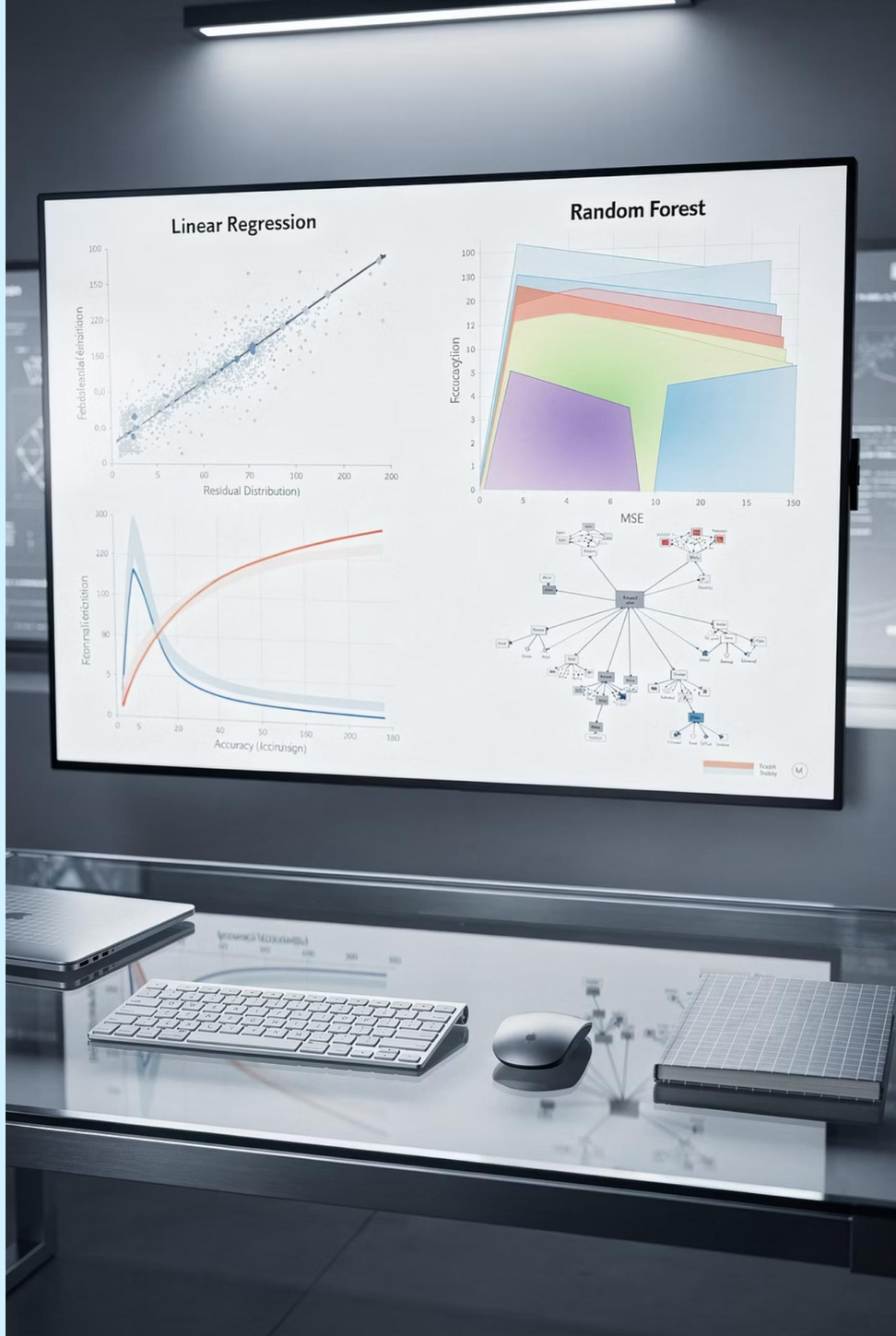
Powtarzalność

Dane testowe są przetwarzane identycznie jak treningowe — poprawna i automatyczna transformacja.

W praktyce projektowej jest to **bardzo dobra technika**, szczególnie przy wdrożeniu modelu na produkcję.

Modele użyte w programie

Przegląd dwóch kluczowych modeli regresyjnych zastosowanych w projekcie analitycznym.



Modele regresyjne

LinearRegression

Najprostszy model regresyjny.

- **Zalety:** prosty, szybki, łatwy do interpretacji
- **Wada:** może nie uchwycić złożonych zależności nieliniowych

RandomForestRegressor

Zaawansowany model oparty na wielu drzewach decyzyjnych.

- **Zalety:** radzi sobie z nieliniowością, lepsze wyniki, odporny na szum
- **Wady:** trudniejszy w interpretacji, cięższy obliczeniowo

Miary oceny modeli

MAE

Średni błąd bezwzględny. Pokazuje przeciętnie, o ile model się myli.

RMSE

Pierwiastek z błędu średniokwadratowego. Mocniej karze duże błędy niż MAE.

R^2

Współczynnik determinacji. Pokazuje, jaka część zmienności zmiennej docelowej jest wyjaśniana przez model.

Interpretacja: im wyższe R^2, tym lepiej. MAE i RMSE powinny być jak najniższe.

Wykres: rzeczywiste vs przewidziane

Dla Random Forest tworzony jest wykres porównujący wartości rzeczywiste (oś X) z przewidzianymi (oś Y).

Idealny model daje punkty leżące blisko prostej **y = x**.

Wykres pozwala ocenić, czy model:

- dobrze odwzorowuje zależności,
- ma duży rozrzut błędów,
- popełnia duże pomyłki dla wysokich wartości.

Segmentacja klientów

Funkcja `customer_segmentation(df, n_clusters=4)` realizuje **uczenie nienadzorowane** — odkrywamy naturalne grupy klientów bez przewidywania etykiet.

Agregacja do poziomu klienta

Liczymy: liczbę zamówień, średnią wartość, łączny przychód, średni rabat, odsetek zwrotów, średni dochód, staż lojalnościowy.

Standaryzacja

Dane są skalowane przed KMeans — konieczne, bo algorytm używa odległości. Bez tego cechy o dużych wartościach zdominowałyby klasteryzację.

KMeans

Grupuje klientów w segmenty: premium, okazjonalni, wysokiego ryzyka zwrotów, lojalni o dużym przychodzie.

Podsumowanie segmentów

Dlaczego analiza średnich jest kluczowa?

Sam numer segmentu nic nie znaczy. Dopiero analiza średnich wartości cech pozwala powiedzieć, kim są klienci w danym klastrze.

Przykładowa interpretacja segmentów:

- **Segment 0:** mało zamówień i niski przychód
- **Segment 1:** wysoki przychód i wysoki dochód
- **Segment 2:** dużo zwrotów

To właśnie jest interpretacja segmentacji — nadanie biznesowego sensu wynikom algorytmu.

Raport końcowy

Funkcja `generate_text_report(...)` zbiera najważniejsze wyniki i zapisuje je do pliku tekstowego. Analiza danych nie kończy się na obliczeniach — wyniki trzeba uporządkować i przedstawić czytelnie.

01

Statystyki ogólne

Liczba zamówień, łączny przychód, średnia wartość zamówienia, najlepsza kategoria i miasto.

02

Analiza i modele

Statystyki opisowe, porównanie grup rabatowych, wyniki modeli regresyjnych.

03

Segmentacja i wnioski

Informacje o segmentach klientów oraz ogólne wnioski analityczne.

Funkcja `main()`

Funkcja sterująca całym programem — wykonuje kolejne etapy we właściwej kolejności.

Dane

Sprawdź plik, wygeneruj lub wczytaj dane, przeprowadź inspekcję.

Przygotowanie

Oczyść dane, utwórz nowe cechy, zapisz oczyszczony zbiór.

Analiza

Wykonaj EDA oraz analizę statystyczną.

Modelowanie

Zbuduj modele, przeprowadź segmentację.

Raport

Wygeneruj końcowy raport tekstowy.

To bardzo dobry przykład organizacji większego programu analitycznego.

Dlaczego ten kod jest dobrym przykładem dydaktycznym?

Ten projekt jest wartościowy, bo pokazuje, że analiza danych to **ciąg logicznie powiązanych etapów**, a nie jedna komenda. Jest znacznie bliższy realnej pracy niż oderwane mini-przykłady.

Programowanie

Organizacja kodu, funkcje, moduły.

Praca z danymi

Czyszczenie, transformacja, inżynieria cech.

Myślenie statystyczne

Metryki, interpretacja, wnioskowanie.

Machine Learning

Regresja, klasteryzacja, ocena modeli.

Najważniejsze rzeczy do zapamiętania

1. Najpierw rozumienie danych, potem model

Nie wolno zaczynać od modelu bez sprawdzenia jakości danych.

2. Dane prawie nigdy nie są idealne

Braki, outliery i błędy to norma, nie wyjątek.

3. Nowe cechy są bardzo ważne

Dobry model często zależy bardziej od sensownych cech niż od samego algorytmu.

4. Interpretacja jest równie ważna jak wynik

Nie wystarczy wygenerować liczbę — trzeba umieć ją wyjaśnić.

5. Dobra organizacja kodu ma znaczenie

Podział na funkcje, katalogi, raporty i wykresy to bardzo dobra praktyka.

Na co zwrócić uwagę podczas samodzielnej pracy?

Analizując ten kod, spróbuj odpowiedzieć na następujące pytania typowe dla praktyki analitycznej:

1. **Które zmienne najbardziej wpływają na `order_value`?**
2. **Czy wysoki rabat zwiększa czy zmniejsza wartość zamówienia?**
3. **Które kategorie produktów są najbardziej problematyczne pod względem zwrotów?**
4. **Czy model liniowy wystarcza, czy model nieliniowy działa lepiej?**
5. **Czy segmenty klientów da się sensownie zinterpretować biznesowo?**
6. **Jak zmienią się wyniki po zmianie sposobu generowania danych?**

Jak można rozbudować ten projekt?

Możliwe rozszerzenia dla bardziej zaawansowanych studentów:

Klasyfikacja zwrotów

Dodaj model klasyfikacyjny przewidujący zwroty.

Analiza sezonowości

Zbadaj wzorce czasowe w danych sprzedażowych.

Feature Importance

Policz ważność cech i zastosuj XGBoost lub Gradient Boosting.

Walidacja krzyżowa

Dodaj CV i porównaj metody imputacji braków oraz PCA przed klasteryzacją.

Dashboard Streamlit

Stwórz interaktywny dashboard prezentujący wyniki analizy.

Wniosek końcowy

> Analiza danych to **proces, a nie pojedyncza funkcja.**

Dobry analityk musi umieć przejść przez cały tok pracy — od surowych danych do zrozumiałych wyników. Ten kod pokazuje właśnie takie pełne podejście:

1. **Przygotuj dane**
   Zrozum strukturę, wykryj problemy.
2. **Zbuduj model**
   Stwórz sensowne cechy, dobierz algorytm.
3. **Oceń i przedstaw**
   Zmierz jakość, wyjaśnij wyniki odbiorcy.